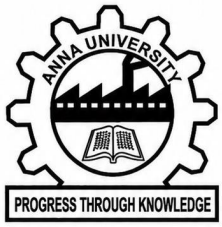
FOOD ORDERING APP – FOODIEGO

A SUMMER INTERNSHIP REPORT

Submitted by

DEEPADHARSHINI M – 810023205071

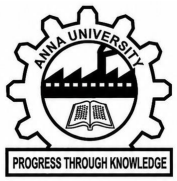


DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),
ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024
DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Food Ordering App – Foodiego"** is the Bonafide work of **DEEPADHARSHINI M (810023205071)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

Foodiego is a cross-platform food and grocery ordering application developed to simplify the way customers discover restaurants, browse cuisines, and place orders online. The system provides a centralized mobile and web platform where customers can sign up or log in using their mobile number (via OTP) or their Google account, browse restaurants by cuisine and location, view top restaurant chains, add items to a cart, place orders, and track order status in real time. Administrators are able to manage restaurant listings, monitor orders, and oversee day-to-day platform activity through a dedicated admin interface.

The application is implemented using Flutter and the Dart programming language, which allows a single codebase to be compiled for Android, iOS, and Web platforms while maintaining a consistent, responsive user interface built from Flutter's widget system. Firebase is used as the backend-as-a-service platform: Firebase Authentication handles secure phone-number (OTP) and Google sign-in, the Firebase Realtime Database stores user, restaurant, cart, and order data, and Firebase Cloud Messaging (FCM) is used to deliver real-time order-status notifications to customers.

One of the main objectives of this project is to demonstrate how a modern, serverless backend can be combined with a declarative UI framework to build a production-style ordering experience without maintaining custom server infrastructure. Key engineering challenges addressed during the internship included structuring the Realtime Database schema for restaurants, menus, carts, and orders; implementing a dual authentication flow that converges on a single Firebase UID; building a live, StreamBuilder-driven UI; and wiring up client-triggered push notifications for cart additions and order-status changes.

Overall, the Foodiego project demonstrates the practical application of Flutter-based mobile/web development combined with Firebase's authentication, database, and messaging services, and highlights how these tools can be integrated to build a complete, real-world food ordering solution, from the customer-facing storefront through to an administrator back-office panel.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 Objectives of the Internship 1.2 Motivation	5 - 6
2	COMPANY PROFILE	6 - 7
3	DAILY WORK REPORT	7 - 9
4	BLOCK DIAGRAM	9
5	FOOD ORDERING APP – FOODIEGO 5.1 Existing System 5.2 Proposed System 5.3 Capabilities	9 - 11
6	PROPOSED METHOD	11 - 12
7	ALGORITHM	12 - 13
8	CODE IMPLEMENTATION	13 - 15
9	SAMPLE OUTPUT	15 - 22
10	ANALYSIS	22 - 23
11	CONCLUSION	23
12	FUTURE WORK	23
13	REFERENCES	23 - 24

1. INTRODUCTION

Foodiego is a full-stack mobile and web application developed to automate and simplify the process of ordering food and groceries from local restaurants and stores. The food delivery industry involves several interconnected operations such as restaurant discovery, cuisine browsing, menu management, cart handling, order placement, payment confirmation, and delivery tracking. In many small and mid-sized food businesses, these operations are still handled manually or through disconnected tools, which leads to slow order processing, miscommunication, and a poor customer experience.

The proposed system provides a centralized platform where customers can register or log in, browse restaurants by cuisine or top restaurant chains, view food categories, search for a specific dish, add items to a cart, place orders, and receive real-time updates on their order status. Administrators are able to manage restaurant and menu information and monitor incoming orders through an administrative interface.

The application follows a client-driven, serverless architecture. The entire user interface — including the sign-in dialog, hero/banner section, location picker, cuisine and food-category sections, restaurant listings, cart, and order summary pages — is built using Flutter widgets in Dart. Firebase acts as the backend, handling authentication, real-time data storage, and push notifications, removing the need for a custom-built server.

1.1 OBJECTIVES OF THE INTERNSHIP

The main objective of the Foodiego application is to develop a digital platform that simplifies and automates the food-ordering experience for both customers and restaurant administrators. The objectives of the project are:

- To provide a fast, intuitive interface for browsing restaurants, cuisines, and food categories.
- To support secure and convenient sign-in through mobile OTP and Google Sign-In.
- To allow customers to add items to a cart and place orders with minimal friction.
- To provide real-time order-status updates using push notifications.
- To give administrators an easy way to manage restaurant data and monitor orders.
- To build the entire application on a single Flutter/Dart codebase deployable to Android, iOS, and Web.
- To gain practical experience integrating a Flutter frontend with Firebase backend services (Auth, Realtime Database, Cloud Messaging).

1.2 MOTIVATION

The growing popularity of online food delivery platforms has changed customer expectations — users now expect to browse menus, place orders, and track deliveries entirely from their phone, in a matter of seconds. Many small food businesses, however, still rely on manual order-taking over phone calls, which is slow, error-prone, and difficult to scale.

The motivation behind this project was to build a lightweight but complete food-ordering platform using modern, widely-used tools — Flutter for a single cross-platform codebase and Firebase for a fully managed backend — and in doing so, gain hands-on experience with authentication flows, real-time databases, state management, and push notifications in a real application.

Building Foodiego also offered practical exposure to the kind of engineering trade-offs a real production team faces — for instance, deciding how authentication state should converge across two different sign-in providers, and how to keep a cart's in-memory state and its Firebase-backed copy consistent without introducing a heavyweight state-management dependency.

2. COMPANY PROFILE

3.

ZIGA INFOTECH VENTURES PVT LTD

About the Company:

Ziga Infotech Ventures Pvt Ltd is an Information Technology (IT) company located in Madipakkam, Chennai, Tamil Nadu, India, operating under the tagline "Success Delivered." The company provides technology consulting and software development services to clients across various business sectors, with a strong focus on mobile-first, cloud-backed application development. It has built a reputation for delivering reliable, user-centric digital solutions by combining modern development frameworks with efficient, serverless backend architectures.

The primary objective of Ziga Infotech Ventures is to develop mobile and web applications that simplify everyday business and consumer processes, while providing a smooth, responsive user experience. The company follows a hands-on, project-driven development approach and emphasizes practical, real-time problem-solving over purely theoretical training, focusing on delivering scalable and maintainable applications using contemporary tools and frameworks.

Apart from client project delivery, the company actively provides offline internship programs for engineering students, offering direct, in-person mentorship under the guidance of experienced project

managers. These programs are designed to bridge the gap between academic learning and industry requirements by placing interns directly onto ongoing mobile application development projects. During the internship, students gain hands-on experience in cross-platform mobile development, backend-as-a-service integration, authentication systems, real-time databases, and the overall mobile app development lifecycle, under the guidance of experienced professionals.

As part of my internship at Ziga Infotech Ventures Pvt Ltd, carried out under the Mobile App Development department, I developed a Food Ordering App – Foodiego using Flutter and Firebase. The project involved designing a responsive, cross-platform user interface, integrating Firebase Authentication for dual sign-in flows, structuring a Firebase Realtime Database schema for restaurants, carts, and orders, and implementing Firebase Cloud Messaging for real-time order-status notifications. This internship helped me improve my technical knowledge, cross-platform development skills, problem-solving abilities, and understanding of a modern, serverless application development lifecycle.

The internship experience also provided practical knowledge of working in a professional software development environment. It enhanced my ability to understand project requirements, develop production-style mobile applications, work with modern development tools, and follow industry practices while building software solutions. Overall, Ziga Infotech Ventures Pvt Ltd provided an excellent platform to apply theoretical concepts in practical situations and strengthen my skills in Flutter and Firebase-based mobile application development.

Company Details:

- Company Name: Ziga Infotech Ventures Pvt Ltd
- Location: Madipakkam, Chennai, Tamil Nadu, India – 600 091
- Department: Mobile App Development
- Internship Tenure: 16 June 2026 – 15 July 2026
- Project Completed During Internship: Food Ordering App – Foodiego (Flutter / Firebase)
- Development Technologies: Flutter, Dart, Firebase Authentication, Firebase Realtime Database, Firebase Cloud Messaging

3. DAILY WORK REPORT

Day 1: Attended orientation session at Ziga Infotech Ventures Pvt Ltd; briefed on internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic – "Food Ordering App – Foodiego" – with the Project Manager.

Day 2: Conducted requirement gathering. Listed the stakeholders (Customer, Administrator) and identified core functional requirements of the food-ordering application.

Day 3: Carried out a literature review of existing food-delivery platforms such as Swiggy, Zomato, and Uber Eats. Documented the drawbacks of manual, phone-call-based ordering for small food businesses.

Day 4: Finalized the scope and objectives of the project. Prepared the hardware and software requirement list, including Flutter SDK, Firebase, and the required IDEs.

Day 5: Set up the development environment – installed Flutter SDK, Visual Studio Code, and Git. Verified installation with a sample Flutter "Hello World" application.

Day 6: Studied Flutter and Dart fundamentals – widget tree structure, StatefulWidget/StatelessWidget, and the pub package manager.

Day 7: Designed the overall system architecture using a layered, client-driven approach (Presentation Layer, Business Logic Layer, Data Layer) with Firebase as the serverless backend.

Day 8: Planned the Firebase Realtime Database schema – identifying nodes for users, restaurants, cart, and orders – and initialized the Firebase project via firebase_options.dart.

Day 9: Implemented Firebase Authentication setup, enabling both Phone (OTP) sign-in and Google Sign-In providers in the Firebase console.

Day 10: Created wireframes for the Home Page, Sign-In dialog, and Cart Page, and planned the responsive Flutter widget layout.

Day 11: Developed hero_section.dart – the home page banner ("Order food & groceries. Delivered to your door.") and the global search entry point.

Day 12: Developed location_picker.dart, a bottom-sheet widget allowing customers to search and select a delivery location.

Day 13: Developed sign_in_page.dart – built the tabbed Login / Sign Up dialog UI with mobile-number and Google Sign-In options.

Day 14: Implemented sendOTP() using Firebase Authentication's verifyPhoneNumber flow, handling the verificationCompleted, verificationFailed, and codeSent callbacks.

Day 15: Implemented _checkUserExistsAndLogin() and the Sign Up flow to create or verify a corresponding user record under users/{uid} in the Realtime Database.

Day 16: Implemented Google Sign-In using the google_sign_in package and GoogleAuthProvider, converging both auth flows on a single Firebase UID.

Day 17: Developed cuisines_section.dart and food_category.dart – horizontally scrollable cuisine and food-category cards on the home page.

Day 18: Developed restaurant_data.dart, defining the MenuItem, Restaurant, and CartItem data models.

Day 19: Developed restaurant_repository.dart, including the watchRestaurants() live stream reading from the restaurants/ node of the Realtime Database.

Day 20: Developed restaurant_list_section.dart and top_restaurant_chains.dart, rendering live restaurant listings using StreamBuilder.

Day 21: Developed food_item_restaurants_page.dart to show restaurants serving a specific searched dish.

Day 22: Developed the CartModel singleton (ChangeNotifier) in restaurant_data.dart, enforcing single-restaurant ordering and mirroring cart additions to Firebase.

Day 23: Developed cart_page.dart, allowing customers to review items and adjust quantities using increment/decrement controls.

Day 24: Developed order_summary_page.dart, including the delivery-address field, bill breakdown, and the _saveOrderToFirebase() function that writes the order to orders/{uid}.

Day 25: Developed order_status_notifier.dart – a listener on onChildChanged that detects genuine order-status transitions.

Day 26: Developed fcm_sender.dart, a client-side Firebase Cloud Messaging sender that posts to the FCM HTTP v1 API for cart and order-status notifications.

Day 27: Developed admin_page.dart – the administrator interface listing all restaurants with options to add, edit, or delete.

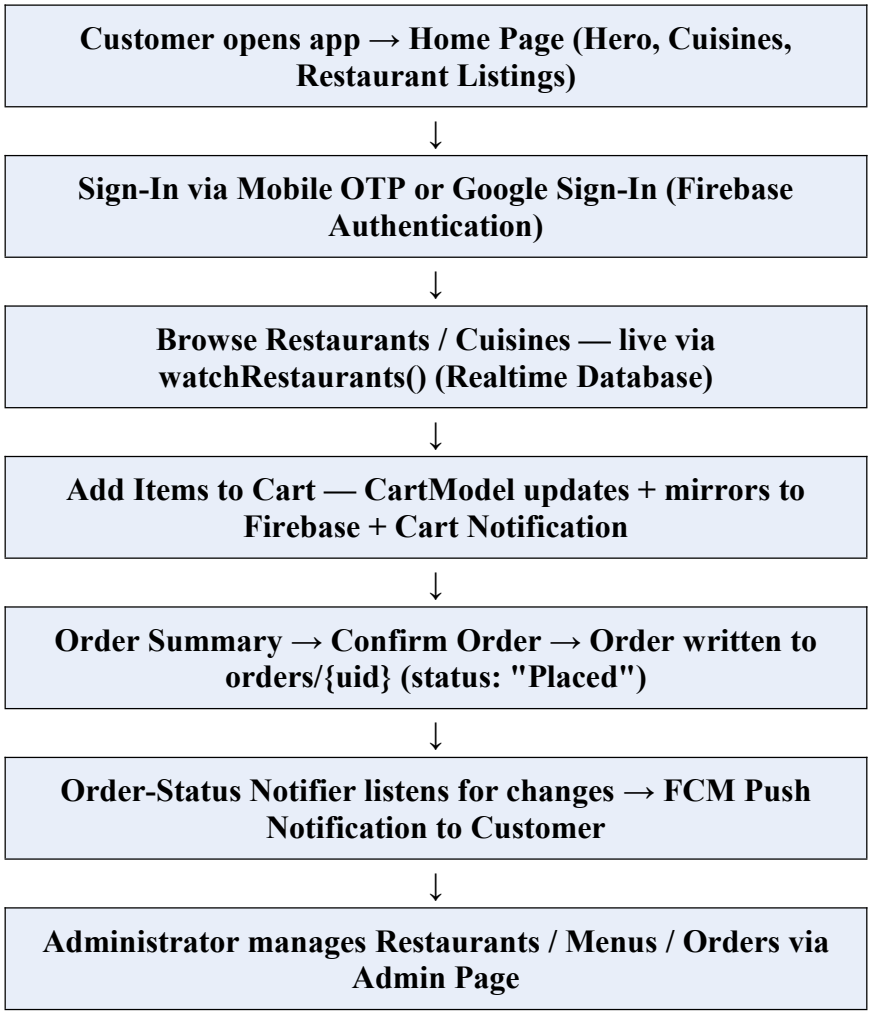
Day 28: Developed custom_appbar.dart and footer.dart, and wired main.dart to route between the customer home screen and the admin panel based on the signed-in account.

Day 29: Performed integration testing across modules – authentication, restaurant browsing, cart, order placement, and notifications – on the Android emulator and Flutter Web (Chrome).

Day 30: Fixed issues identified during testing, took screenshots for documentation, and compiled the final internship report.

4. BLOCK DIAGRAM

Foodiego follows a client-driven, serverless architecture in which the Flutter application communicates directly with Firebase services — there is no custom backend server process. The workflow below illustrates the flow of control between the Customer, the Flutter/Firebase client, and the Administrator.



5. FOOD ORDERING APP – FOODIEGO

5.1 Existing System

Traditional food-ordering processes for small and mid-sized food businesses mainly depend on phone-call-based ordering, manual note-taking of customer orders, word-of-mouth or printed menus, and have no centralized system for tracking order status.

This existing approach suffers from several limitations:

- High dependency on manual, phone-call-based communication for taking orders.
- Increased chance of order errors due to manual note-taking.
- No live visibility into order status for customers once an order is placed.
- Difficult to scale as order volume grows, since everything routes through a single phone line.
- Poor visibility for administrators into incoming orders across restaurants.
- Inconsistent experience across different devices and platforms.

5.2 Proposed System

The proposed Foodiego system combines customer authentication (phone OTP and Google Sign-In), restaurant, cuisine, and menu browsing sourced live from Firebase, cart and order management, real-time order-status notifications via Firebase Cloud Messaging, and an administrative interface for restaurant and order management.

Modern food-delivery platforms such as Swiggy, Zomato, and Uber Eats demonstrate that mobile-first, cloud-backed ordering systems significantly improve customer convenience and operational efficiency. Combining a reactive UI framework (Flutter) with a managed cloud backend (Firebase Authentication, Realtime Database, and Cloud Messaging) reduces development effort while still delivering a responsive, real-time user experience — which is the approach Foodiego adopts.

5.3 Capabilities

- User Management – secure sign-in via mobile OTP or Google Sign-In, converging on a single Firebase UID.
- Restaurant & Cuisine Browsing – live restaurant listings, cuisine categories, and top restaurant chains read from the Realtime Database.
- Dish Search – search for a specific food item across restaurants.
- Cart Management – add items to a cart restricted to a single restaurant at a time, with live totals.
- Order Placement & Tracking – place orders and receive real-time order-status push notifications.
- Admin Panel – add, edit, or delete restaurants and menu items, and monitor orders.

- Cross-Platform Delivery – a single Flutter/Dart codebase deployable to Android, iOS, and Web.
- Removes dependency on manual, phone-based ordering and reduces server-maintenance overhead by relying on managed Firebase services.

6. PROPOSED METHOD

6.1 User Authentication

- Handles sign-in via mobile OTP verification (verifyPhoneNumber) and Google Sign-In (GoogleAuthProvider).
- Checks the Realtime Database for an existing users/{uid} record; creates one on sign-up, or signs the user back out with an error if none exists on login.
- Both authentication flows converge on a single Firebase UID, keeping the customer's identity consistent regardless of sign-in method.

6.2 Restaurant & Cuisine Browsing

- restaurant_repository.dart exposes watchRestaurants(), a live stream over the restaurants/ node.
- restaurant_list_section.dart and top_restaurant_chains.dart render this stream via StreamBuilder so listings update the instant data changes.
- cuisines_section.dart and food_category.dart provide horizontally scrollable category filters on the home page.

6.3 Cart Management

- CartModel (a ChangeNotifier singleton) holds in-memory cart items and notifies every listening widget — the cart badge, cart page, and order summary — automatically.
- canAddFrom() enforces that a cart may only contain items from a single restaurant at a time.
- The first addition of a distinct item mirrors the entry to cart/{uid}/{itemId} in Firebase and fires a local "Added to cart" push notification.

6.4 Order Management

- order_summary_page.dart collects the delivery address and computes subtotal, delivery fee, tax, and grand total.
- Confirming the order writes a new record under orders/{uid}/{orderId} with an initial status of "Placed".
- order_status_notifier.dart listens for onChildChanged events on the order and pushes a notification only on a genuine status transition.

6.5 Admin Management

- admin_page.dart streams the same live restaurant list used on the customer side.
- Administrators can add, edit, or delete a restaurant and its menu items through AdminRestaurantEditPage.
- Any change made in the admin panel is reflected everywhere immediately, since both sides read from the same Firebase stream.

7. ALGORITHM

7.1 Mobile OTP Sign-In

- Customer enters a 10-digit mobile number and taps Send OTP.
- App validates the number length, then calls Firebase's verifyPhoneNumber().
- On codeSent, the OTP field is shown; on verificationCompleted, the credential signs the user in automatically.
- _checkUserExistsAndLogin() checks users/{uid}: if a record exists, login succeeds; otherwise the user is signed back out with "No account found."

7.2 Google Sign-In

- Customer taps Continue with Google; the google_sign_in package opens the account picker.
- The returned Google credential is exchanged for a Firebase credential via GoogleAuthProvider.
- _handleGoogleUser() checks or creates the corresponding users/{uid} record, converging on the same UID used by OTP sign-in.

7.3 Add Item to Cart

- Customer taps ADD on a menu item; canAddFrom() checks the cart is empty or already ordering from the same restaurant.
- If the item already exists in the cart, its quantity is incremented; otherwise a new CartItem is created.
- notifyListeners() refreshes the cart badge/UI; on a first-time add, the item is mirrored to Firebase and a cart notification is sent.

7.4 Place Order

- Customer reviews the cart, enters a delivery address on the Order Summary page, and confirms the order.
- The order (items, address, subtotal, delivery fee, tax, total) is written to orders/{uid}/{orderId} with status "Placed".
- An "order placed" push notification is sent, and OrderStatusNotifier begins tracking the order for further status changes.

7.5 Order-Status Notification

- OrderStatusNotifier listens on orders/{uid} for onChildChanged events.
- For each change, the new status is compared against the last known status for that order ID.
- If the status has genuinely changed (and it is not the initial snapshot), FcmSender pushes a notification.

7.6 SYSTEM DESIGN AND METHODOLOGY

The system is designed using a layered, client-driven architecture. The application is divided into the following layers:

Presentation Layer

Handles user interaction through Flutter widgets — the hero section, sign-in dialog, location picker, cuisine and food-category sections, restaurant listings, cart, and order summary pages.

Business Logic Layer

Implemented within Dart classes and repositories (e.g. RestaurantRepository, CartModel, OrderStatusNotifier) that process application rules such as checking whether a user account exists, calculating cart totals, restricting a cart to a single restaurant at a time, and reacting to order-status changes.

Data Layer

Firebase Authentication and the Firebase Realtime Database are responsible for storing and managing user accounts, restaurant data, cart entries, and order records. Firebase Cloud Messaging delivers real-time notifications.

The development methodology follows the Software Development Life Cycle (SDLC):

- Requirement Analysis
- System Design
- Development
- Testing
- Deployment
- Maintenance

8.CODE IMPLEMENTATION

8.0.1 Frontend Implementation

The frontend is developed entirely in Flutter using Dart. Because Flutter compiles to native Android, iOS, and Web targets from a single codebase, the same widget tree drives the experience across platforms.

Key frontend screens/widgets include:

- Sign-In Dialog (Login / Sign Up tabs, OTP fields, Google Sign-In button).

- Home Page (Custom App Bar, Hero Section, Location Picker, Food Categories, Top Restaurant Chains, Cuisines, Restaurant Listings, Footer).
- Food Item Restaurants Page (search-by-dish results).
- Cart Page and Order Summary Page.
- Admin Home Page and Admin Restaurant Edit Page.

State is managed using Flutter's built-in StatefulWidget / setState pattern for local UI state (e.g. OTP entry, loading spinners, form fields), while StreamBuilder is used throughout to reflect live Firebase Realtime Database updates directly in the UI without any additional state-management package. The cart itself is modeled as a small ChangeNotifier singleton (CartModel.instance) so that the cart icon badge, the cart page, and the order summary page all stay in sync automatically.

8.0.2 Backend Implementation (Firebase)

The backend of Foodiego is implemented entirely using Firebase's managed services rather than a custom server.

Firebase Authentication

Handles two sign-in flows: phone-number verification with OTP (using verifyPhoneNumber and PhoneAuthProvider) and Google Sign-In (using the google_sign_in package together with GoogleAuthProvider). Both flows converge on a single Firebase UID used to key the user's record in the database, so a customer's identity is consistent regardless of which method they used to sign in.

Firebase Realtime Database

Stores user profiles, restaurant and menu data, cart entries, and order records as nested JSON under paths such as users/{uid}, restaurants/{restaurantId}, cart/{uid}/{itemId}, and orders/{uid}/{orderId}. The app reads this data using both one-time get()/once() calls (e.g. checking whether a user exists) and live onValue / onChildChanged listeners (e.g. restaurant listings and order-status tracking).

Firebase Cloud Messaging (FCM)

Used to push real-time notifications to the customer's device for two events — items added to the cart, and order-status changes — coordinated through the OrderStatusNotifier and FcmSender modules. FcmSender authenticates against the FCM HTTP v1 API directly from the client using a bundled service-account credential, which keeps the project simple for a learning/internship setting, though in a production deployment this responsibility would be moved to a trusted server or Cloud Function.

8.1 Mobile OTP Sign-In — sign_in_page.dart

Sends an OTP to the user’s mobile number using Firebase Authentication’s phone-verification flow.

```
Future<void> sendOTP() async {
  final phone = mobileController.text.trim();
  if (phone.length != 10) {
    _showError('Enter a valid 10-digit mobile number');
    return;
  }
  await _auth.verifyPhoneNumber(
```

```
    phoneNumber: '+91$phone',
    verificationCompleted: (credential) async {
      await _auth.signInWithCredential(credential);
      await _checkUserExistsAndLogin();
    },
    verificationFailed: (e) => _showError(e.message ?? 'Verification failed.'),
    codeSent: (verificationId, resendToken) {
      setState(() { _verificationId = verificationId; isOtpSent = true; });
    },
    codeAutoRetrievalTimeout: (verificationId) => _verificationId = verificationId,
  );
}
```

8.2 Checking / Creating the User Record — sign_in_page.dart

After successful authentication, the app checks whether a profile already exists for the signed-in Firebase UID before allowing login.

```
Future<void> _checkUserExistsAndLogin() async {
  final user = _auth.currentUser;
  final ref = _database.ref('users/${user!.uid}');
  final snapshot = await ref.get();
  if (snapshot.exists) {
    _showMessage('Login successful!');
    Navigator.pop(context);
  } else {
    await _auth.signOut();
    _showError('No account found. Please sign up first.');
  }
}
```

8.3 Cart Model — restaurant_data.dart

A lightweight, app-wide singleton cart built as a ChangeNotifier, so any widget that listens to it rebuilds automatically when items change.

```
class CartModel extends ChangeNotifier {
  static final CartModel instance = CartModel._internal();
  final Map<String, CartItem> _items = {};

  bool canAddFrom(String restaurantName) =>
      _items.isEmpty || _items.values.first.restaurantName == restaurantName;
```

```
  void addItem({required String id, required String name,
      required double price, required String restaurantName}) {
    final isNewItem = !_items.containsKey(id);
    if (!isNewItem) {
      _items[id]!.quantity++;
    } else {
      _items[id] = CartItem(id: id, name: name, price: price,
          restaurantName: restaurantName);
    }
    notifyListeners();
    if (isNewItem) {
      _pushCartItemToFirebase(id: id, name: name, price: price);
      _sendCartNotification(itemName: name);
    }
  }
}
```

8.4 Live Restaurant Data — restaurant_repository.dart

Restaurant and menu data is streamed live from the Firebase Realtime Database, so any screen using this stream updates automatically the moment data changes.

```
Stream<List<Restaurant>> watchRestaurants() {
  return _ref.onValue.map((event) {
    final raw = event.snapshot.value;
    if (raw == null) return <Restaurant>[];
    final map = Map<dynamic, dynamic>.from(raw as Map);
    return map.entries
        .where((e) => e.value is Map)
        .map((e) => Restaurant.fromMap(e.key.toString(),
            Map<dynamic, dynamic>.from(e.value as Map)))
        .toList();
  });
}
```

8.5 Placing an Order — order_summary_page.dart

Writes the confirmed order to Firebase, then fires an “order placed” push notification.

```
Future<String?> _saveOrderToFirebase() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null || _cart.items.isEmpty) return null;
  final ref = _ordersRef('orders/${user.uid}').push();
  await ref.set({
    'restaurantName': _cart.items.first.restaurantName,
```

```
    'deliveryAddress': _addressController.text.trim(),
    'subtotal': _subtotal, 'deliveryFee': _deliveryFee,
    'tax': _tax, 'total': _grandTotal,
    'status': 'Placed', 'placedAt': DateTime.now().toIso8601String(),
  });
  _sendOrderPlacedNotification(restaurantName: _cart.items.first.restaurantName);
  return ref.key;
}
```

8.6 Order-Status Notifications — order_status_notifier.dart

Watches orders/{uid}/ for status changes and fires a push notification only when a status genuinely changes.

```
void start() {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;
  _ordersRef = _db.ref('orders/${user.uid}');
  _ordersRef!.onChildChanged.listen((event) {
    final orderId = event.snapshot.key;
    final data = event.snapshot.value;
    if (orderId == null || data is! Map) return;
    final newStatus = data['status']?.toString();
    final previousStatus = _lastKnownStatus[orderId];
    _lastKnownStatus[orderId] = newStatus!;
    if (previousStatus != null && previousStatus != newStatus) {
      _notify(newStatus);
    }
  }
```

8.7 Application Entry Point and FCM Token Registration — main.dart

Initializes Firebase, requests notification permission, saves the device's FCM token against the signed-in user, and starts the order-status listener as soon as a session is available.

```
 void main() async {
    WidgetsFlutterBinding.ensureInitialized();
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
    );

    // --- Firebase Cloud Messaging setup ---
    final messaging = FirebaseMessaging.instance;
    await messaging.requestPermission();

    final token = await messaging.getToken();
```

```
  await _saveTokenForCurrentUser(token);

  FirebaseAuth.instance.authStateChanges().listen((user) async {
    final currentToken = await FirebaseMessaging.instance.getToken();
    await _saveTokenForCurrentUser(currentToken);
    if (user != null) {
      OrderStatusNotifier.instance.start()
}});
```

8.8 Routing Admin vs. Customer Home — main.dart

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: StreamBuilder<User?>(
        stream: FirebaseAuth.instance.authStateChanges(),
        builder: (context, snapshot) {
          final user = snapshot.data;

          // Admin account signed in -> show the admin panel instead
          // of the normal customer-facing home screen.
          if (user != null && user.email == 'admin@gmail.com') {
            return const AdminHomePage();
          }
          return const _CustomerHome();
        },
      ),
    );
  }
}
```

8.9 Admin Panel — Live Restaurant List — admin_page.dart

The admin home page streams the same restaurant list used on the customer side, so any change made in the admin panel is reflected everywhere immediately.

```
class AdminHomePage extends StatelessWidget {
  const AdminHomePage({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Admin — Restaurants')),
      body: StreamBuilder<List<Restaurant>>(
        stream: RestaurantRepository.instance.watchRestaurants(),
        builder: (context, snapshot) {
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final restaurants = snapshot.data!;
          return ListView.builder(
            itemCount: restaurants.length,
            itemBuilder: (context, i) => ListTile(
              title: Text(restaurants[i].name),
              subtitle: Text(restaurants[i].location),
              onTap: () => Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => AdminRestaurantEditPage(
                    restaurant: restaurants[i],
                  ),
                ),
              ),
            ),
          );
        },
      ),
    );
  }
}
```

11.10 Sending a Push Notification — fcm_sender.dart

A direct, client-side sender that authenticates with a bundled Firebase service-account key and posts to the FCM HTTP v1 API — used both for cart notifications and order-status notifications.

```
class FcmSender {
  FcmSender._internal();
  static final FcmSender instance = FcmSender._internal();

  static const _projectId = 'food-ordering-app-5c822';
  static const _scopes = ['https://www.googleapis.com/auth/firebase.messaging'];

  auth.AutoRefreshingAuthClient? _client;

  Future<void> sendNotification({
    required String token,
    required String title,
```

```
    required String body,
  }) async {
    try {
      final client = await _getClient();
      final url = Uri.parse(
        'https://fcm.googleapis.com/v1/projects/$_projectId/messages:send',
      );
      final payload = {
        'message': {
          'token': token,
          'notification': {'title': title, 'body': body},
        },
      };
      final response = await client.post(
        url,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode(payload),
      );
      if (response.statusCode != 200) {
        print('FCM push failed: ${response.statusCode} ${response.body}');
      }
    } catch (e) {
      print('FCM push error: $e');
    }
  }
}
```

Fig 11.10 — For a production deployment this logic and its service-account key would move server-side (e.g. into a Cloud Function) rather than shipping inside the client app.

9. SAMPLE OUTPUT

9.1 Popular Restaurants — Full Listing

Displays the full restaurant grid on the home page — each card shows a food image, restaurant name, locality, rating, delivery time, and any active offer, rendered live via StreamBuilder against the restaurants/ node.The Foodiego home page greets guests with a bold hero banner (“Order food & groceries. Delivered to your door.”), the current delivery location (Nandambakkam, Chennai, Tamil Nadu), a global search bar, a horizontally scrollable “What's on your mind?” quick-pick strip (Idli, Dosa, Pongal, Vada, Coffee), and a “Top restaurant chains in Chennai” section showing live restaurant cards — each with its offer tag, rating, delivery time, and locality — pulled from the Realtime Database via restaurant_list_section.dart.

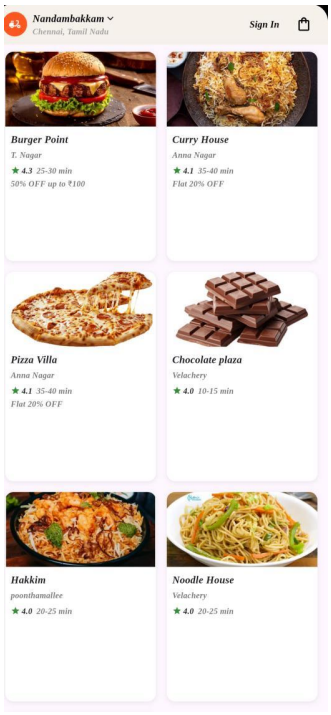


Fig 9.1 — Popular Restaurants — Full Listing

9.2 Welcome Dialog — Sign Up Tab

The Sign Up tab collects Full Name, Email, and Mobile Number before sending an OTP; once verified, a corresponding user record is created under users/ in the Realtime Database.

Tapping Sign In opens the Welcome dialog implemented in sign_in_page.dart. The Login tab accepts a mobile number and triggers Send OTP; an alternate “Login with email instead” link and a “Continue with Google” button are also provided, giving customers a choice between phone-OTP and Google Sign-In flows that converge on a single Firebase UID.

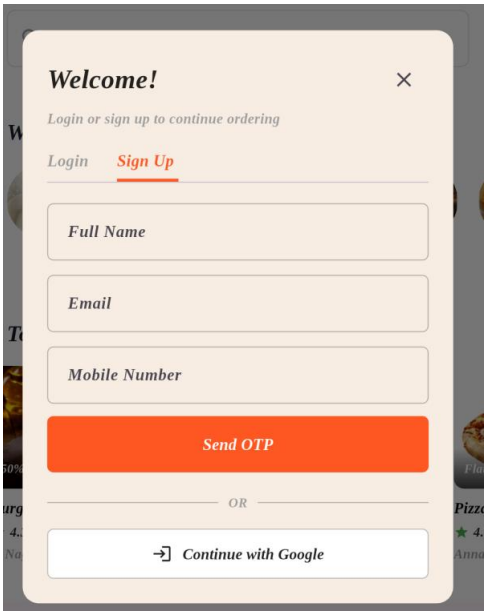


Fig 9.2 — Welcome Dialog — Sign Up Tab

9.3 Login with Email and Password

For accounts created with an email and password — including the reserved admin account — the Login tab accepts an email address and password, with a link to switch back to the OTP flow.

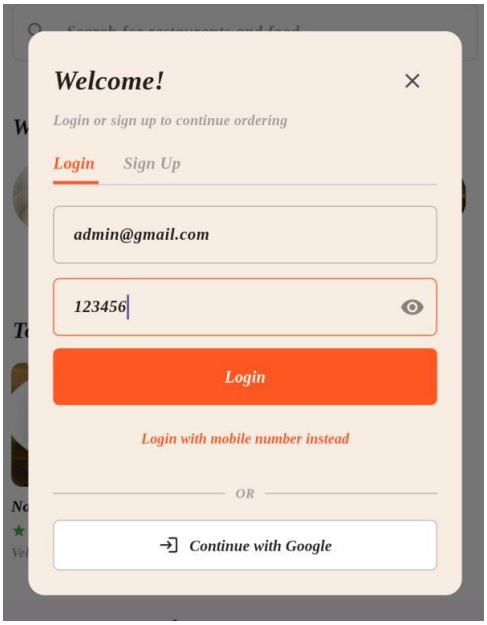


Fig 9.3 — Login with Email and Password

9.4 Restaurants to Explore (with Filters)

A horizontally scrollable filter bar — All, Fast Delivery, Rating 4.0+, Pure Veg, Offers — sits above the restaurant grid, letting customers narrow results without leaving the home page.

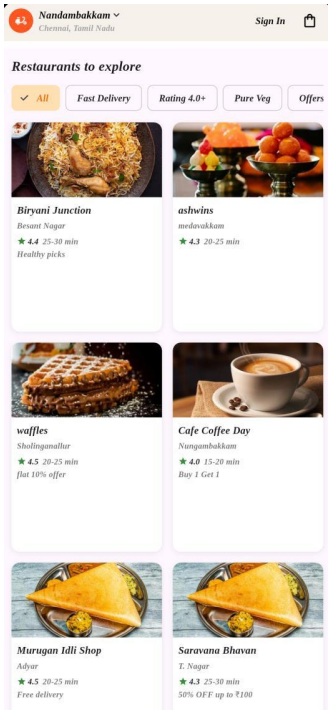


Fig 9.4 — Restaurants to Explore (with Filters)

9.5 Cuisine Listing (South Indian)

Selecting a cuisine card navigates to a dedicated listing screen showing every restaurant tagged under that cuisine, each with its rating, delivery time, and offer tag.

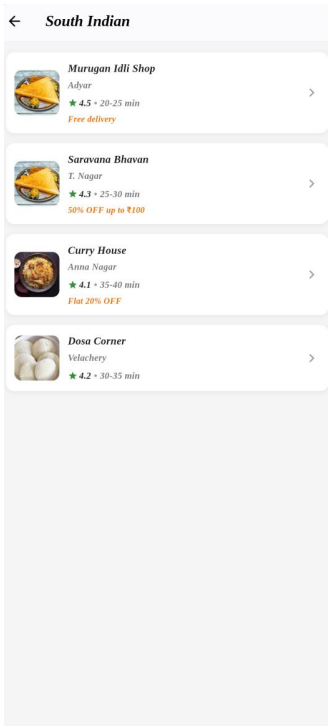


Fig 9.5 — Cuisine Listing (South Indian)

9.6 My Cart

The My Cart page lists every item added from the current restaurant, with per-item increment/decrement steppers and a running total, before the customer proceeds to Place Order.



Fig 9.6 — My Cart

9.7 Order Summary and Checkout

Presents the delivery address, the itemised order, and a bill-details breakdown — subtotal, delivery fee, and taxes — culminating in the amount to pay and a Confirm Order button.

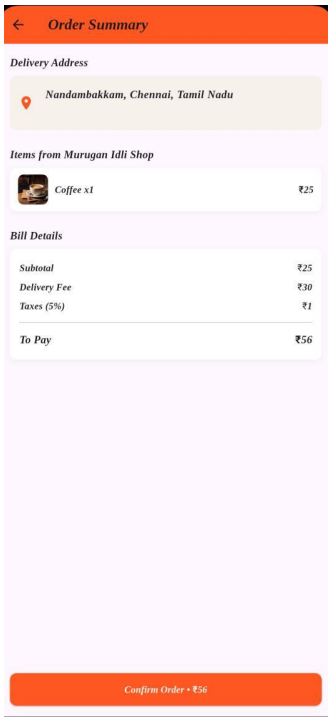


Fig 9.7 — Order Summary and Checkout

9.8 Admin — Manage Restaurants

Signing in with the admin account routes to the admin panel, which lists every restaurant with its locality and menu-item count, and provides options to add or edit a restaurant.

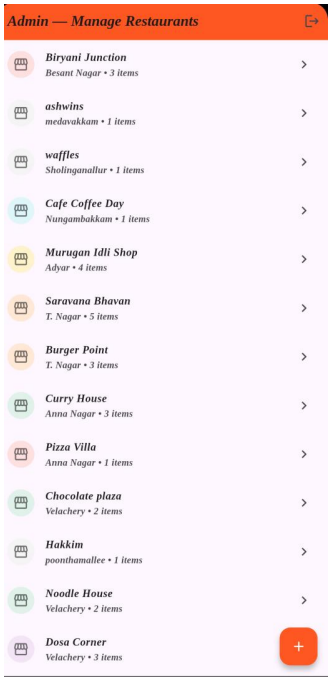


Fig 9.8 — Admin — Manage Restaurants

9.9 Admin — Edit Restaurant

The Edit Restaurant screen pre-fills all fields for an existing restaurant and allows the admin to edit or delete any menu item, or persist changes with Save Changes.

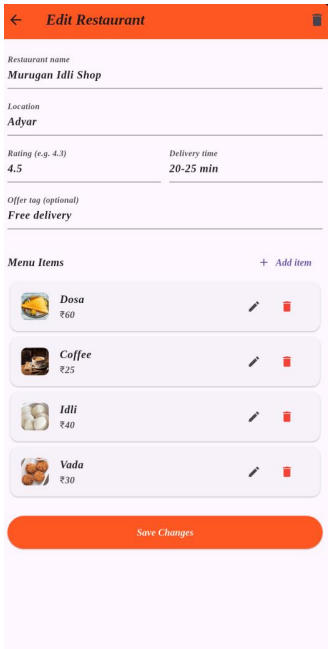


Fig 9.9 — Admin — Edit Restaurant

9.10 Home Page (Guest View)

The Foodiego home page greets guests with a bold hero banner, the current delivery location, a global search bar, a horizontally scrollable "What's on your mind?" quick-pick strip, and a "Top restaurant chains" section with live restaurant cards.

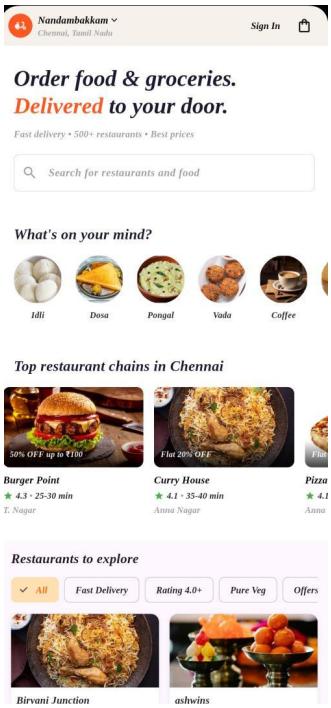


Fig 9.10 — Home Page (Guest View)

9.11 Welcome Dialog — Login Tab

Tapping Sign In opens the Welcome dialog. The Login tab accepts a mobile number and triggers Send OTP; an alternate "Login with email instead" link and a "Continue with Google" button are also provided.

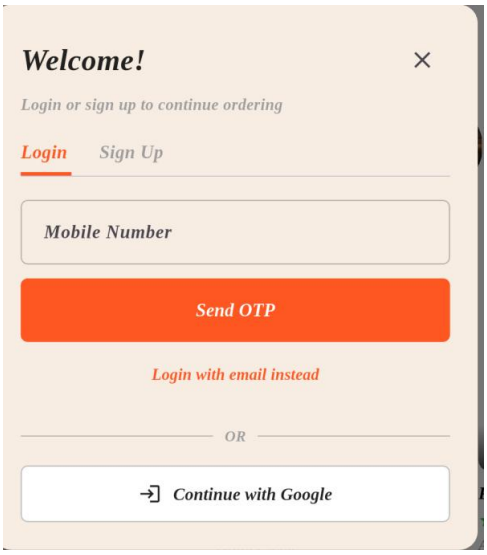


Fig 9.11 — Welcome Dialog — Login Tab

9.12 Cuisine Listing (Fast Food)

The equivalent Fast Food cuisine listing surfaces the matching restaurants. Both cuisine screens reuse the same list-tile layout and navigate into the shared restaurant-menu page on tap.

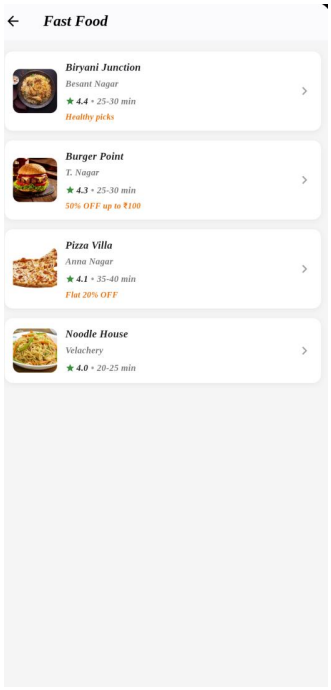


Fig 9.12 — Cuisine Listing (Fast Food)

9.13 Restaurant Menu Page

The restaurant-detail page shows a banner image, the restaurant's rating, delivery time, and locality, an offer badge, and the full menu, each item with its price and an ADD button that pushes it into the shared CartModel.

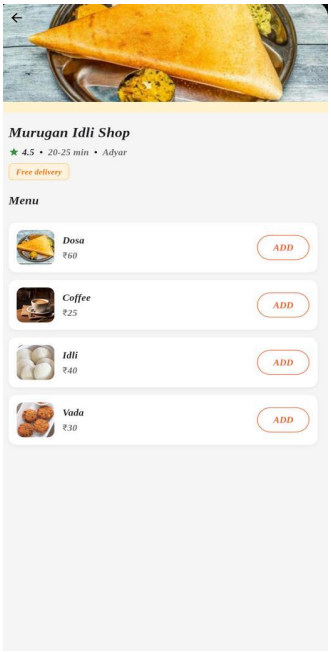


Fig 9.13 — Restaurant Menu Page

9.14 Admin — Add Restaurant

The Add Restaurant form collects the restaurant name, location, rating, delivery time, and an optional offer tag, along with an initially empty menu-items list populated via Add item before saving.

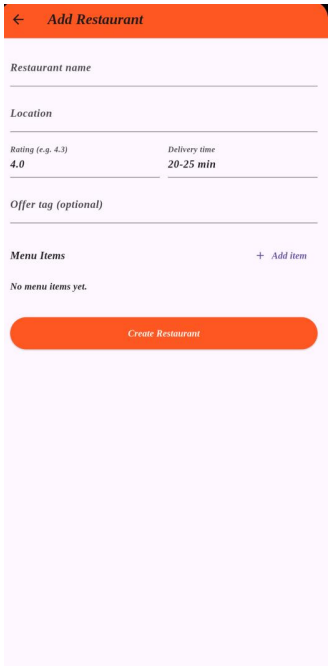


Fig 9.14 — Admin — Add Restaurant

10. ANALYSIS

Testing of the Foodiego application was carried out module by module on both the Android emulator and Flutter Web (Chrome), covering authentication, restaurant browsing, cart management, and order placement.

Functional analysis confirmed that both mobile OTP registration/login and Google Sign-In behaved correctly, including edge cases such as an unregistered number, an invalid OTP, and a cancelled Google sign-in attempt — each of which failed gracefully with a clear, user-facing message rather than crashing the app.

The cart's single-restaurant restriction was verified by attempting to add an item from a second restaurant, which was correctly blocked. Order placement was confirmed to create a record under orders/{uid} with an initial "Placed" status, and a status change made directly in the Firebase console was confirmed to trigger a push notification while the app was open.

Because restaurant listings and order-status updates are driven by StreamBuilder and onValue/onChildChanged listeners rather than one-time fetches, the UI was observed to reflect Firebase

changes almost instantly across both platforms, validating the choice of a live, client-driven architecture over a polling-based approach.

Usability observations during testing indicated that the single-restaurant cart restriction and the clear "No account found" / "Invalid OTP" error messaging reduced user confusion during sign-up and ordering, while the Firebase-managed backend meant no server-side scaling or maintenance work was required to support the tested load.

11. CONCLUSION

The Foodiego application successfully demonstrates the use of Flutter and Firebase to build a modern, cross-platform food-ordering system. The application provides a smooth authentication experience through both mobile OTP and Google Sign-In, live restaurant and menu browsing, cart and order management, and real-time order-status notifications.

The project reduces the manual effort involved in traditional phone-based ordering, improves data consistency through a centralized Firebase backend, and provides a consistent user experience across Android, iOS, and Web from a single Dart codebase. It also provided practical, hands-on experience with authentication flows, real-time databases, and push-notification systems in a production-style mobile application, as well as experience building and maintaining a lightweight administrative back office within the same codebase.

12. FUTURE WORK

- Integration of a real online payment gateway (UPI, cards, wallets).
- Delivery-partner assignment and live GPS-based order tracking.
- Ratings and reviews for restaurants and menu items.
- AI-based restaurant and dish recommendations.
- Migration/expansion to Cloud Firestore for more advanced querying.
- Moving order-status change detection into a server-side Cloud Function so notifications also work when the app is fully closed.
- Admin analytics dashboard with sales and order-trend reporting.
- Multi-language support.
- iOS App Store and Web production deployment.

13. REFERENCES

1. Flutter Official Documentation – https://docs.flutter.dev

2. Dart Language Documentation – https://dart.dev/guides

3. FlutterFire (Firebase for Flutter) Documentation – https://firebase.flutter.dev

4. Firebase Authentication Documentation – https://firebase.google.com/docs/auth

5. Firebase Realtime Database Documentation – https://firebase.google.com/docs/database

6. Firebase Cloud Messaging Documentation – https://firebase.google.com/docs/cloud-messaging

7. google_sign_in package – https://pub.dev/packages/google_sign_in

8. GeeksforGeeks – Flutter and Firebase integration tutorials